

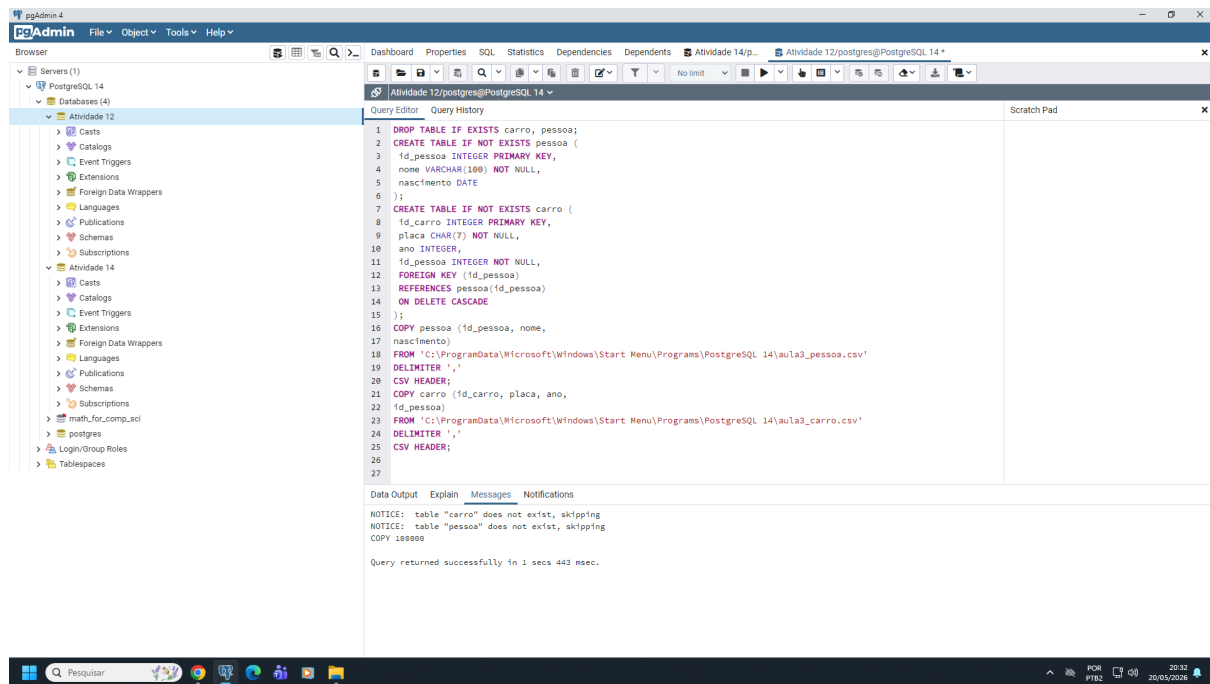
ATIVIDADE DA AULA 12 - Banco de Dados - Relacional (2º Semestre)

ALUNA:Nadla Fernandes Ferreira

RA:2581392613046

1º Comando com os caminhos dos arquivos:

```
DROP TABLE IF EXISTS carro, pessoa;
CREATE TABLE IF NOT EXISTS pessoa (
    id_pessoa INTEGER PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    nascimento DATE
);
CREATE TABLE IF NOT EXISTS carro (
    id_carro INTEGER PRIMARY KEY,
    placa CHAR(7) NOT NULL,
    ano INTEGER,
    id_pessoa INTEGER NOT NULL,
    FOREIGN KEY (id_pessoa)
    REFERENCES pessoa(id_pessoa)
    ON DELETE CASCADE
);
COPY pessoa (id_pessoa, nome,
nascimento)
FROM 'C:\ProgramData\Microsoft\Windows\Start
Menu\Programs\PostgreSQL 14\aula3_pessoa.csv'
DELIMITER ','
CSV HEADER;
COPY carro (id_carro, placa, ano,
id_pessoa)
FROM 'C:\ProgramData\Microsoft\Windows\Start
Menu\Programs\PostgreSQL 14\aula3_carro.csv'
DELIMITER ','
CSV HEADER;
```

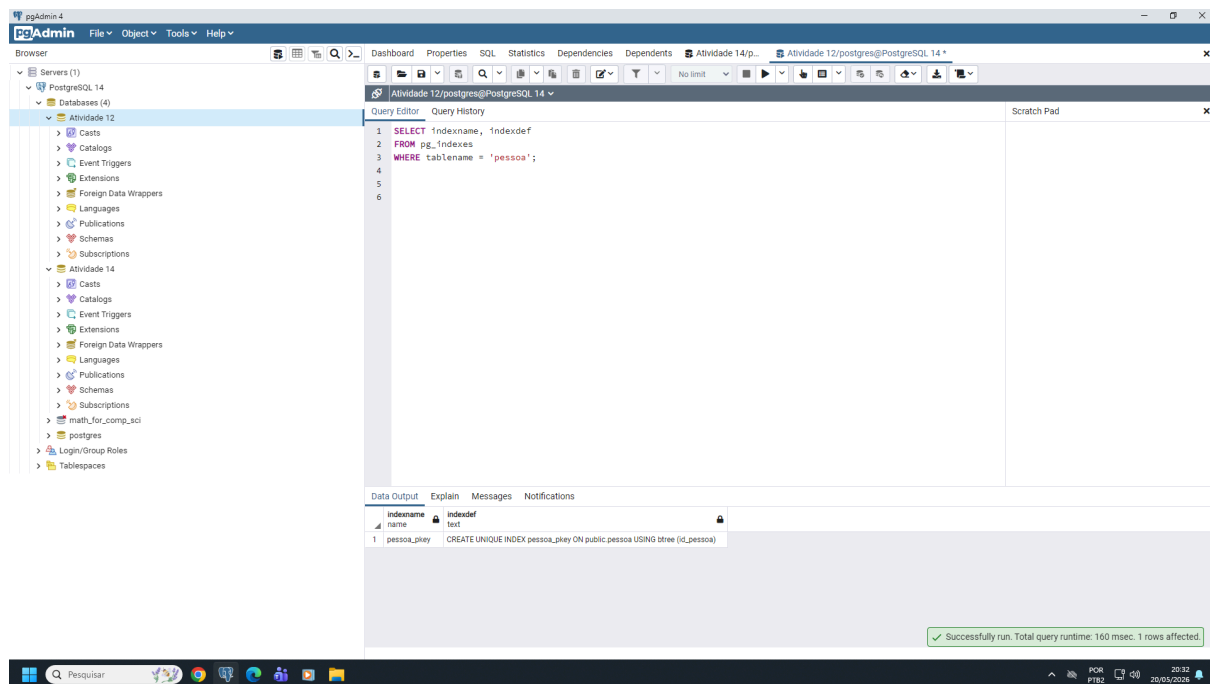


2º Comando para listar os índices de uma tabela:

```

SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'pessoa';

```



EXERCÍCIO 1

PARTE A:

1º Execute a consulta abaixo e registre o tipo de varredura e o tempo retornado:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nome = 'Ana Silva';
```

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 14' and then to 'Atividade 12'. The 'Query Editor' is active, displaying the SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM pessoa
4 WHERE nome = 'Ana Silva';
5
6
```

. Below the query editor, the 'Query Plan' is visible, showing the execution details:

```
1 Seq Scan on pessoa (cost=0.00..1915.00 rows=4 width=23) (actual time=2.957.7114 rows=5 loops=1)
2 [...] Filter: ((nome)::text = 'Ana Silva'::text)
3 [...] Rows Removed by Filter: 99995
4 Planning Time: 0.952 ms
5 Execution Time: 7.127 ms
```

. At the bottom right, a green status bar indicates: 'Successfully run. Total query runtime: 47 msec. 5 rows affected.'

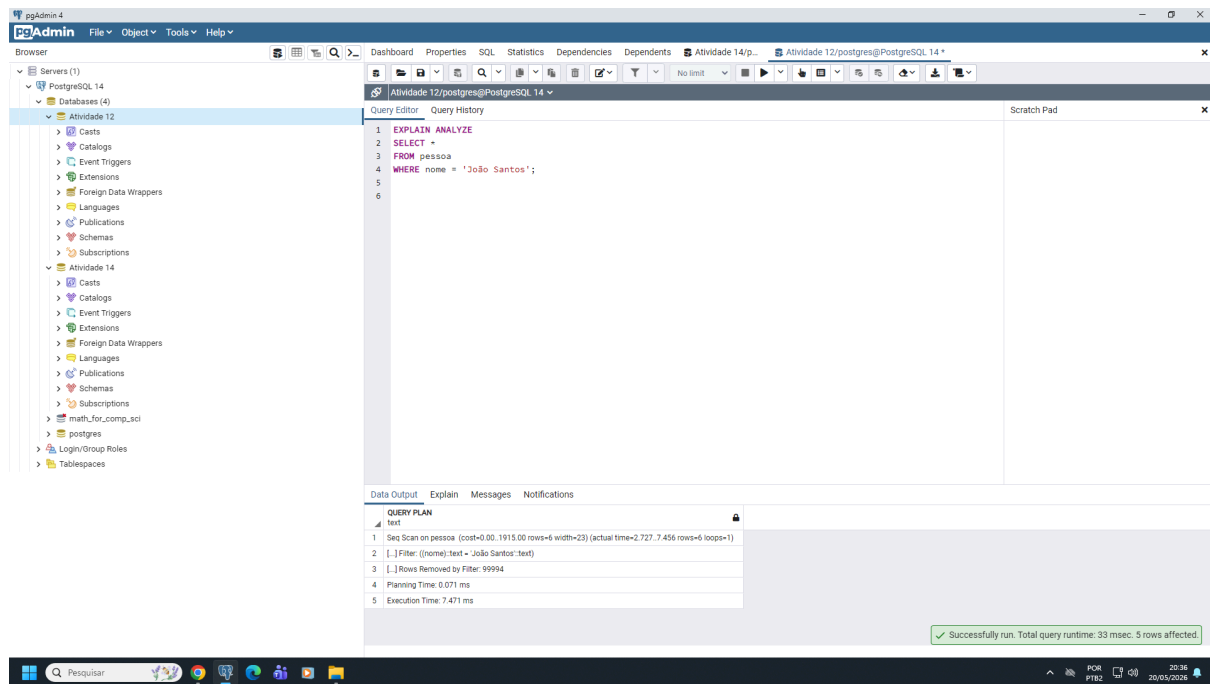
2º Execute uma segunda vez com outro nome comum, por exemplo:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

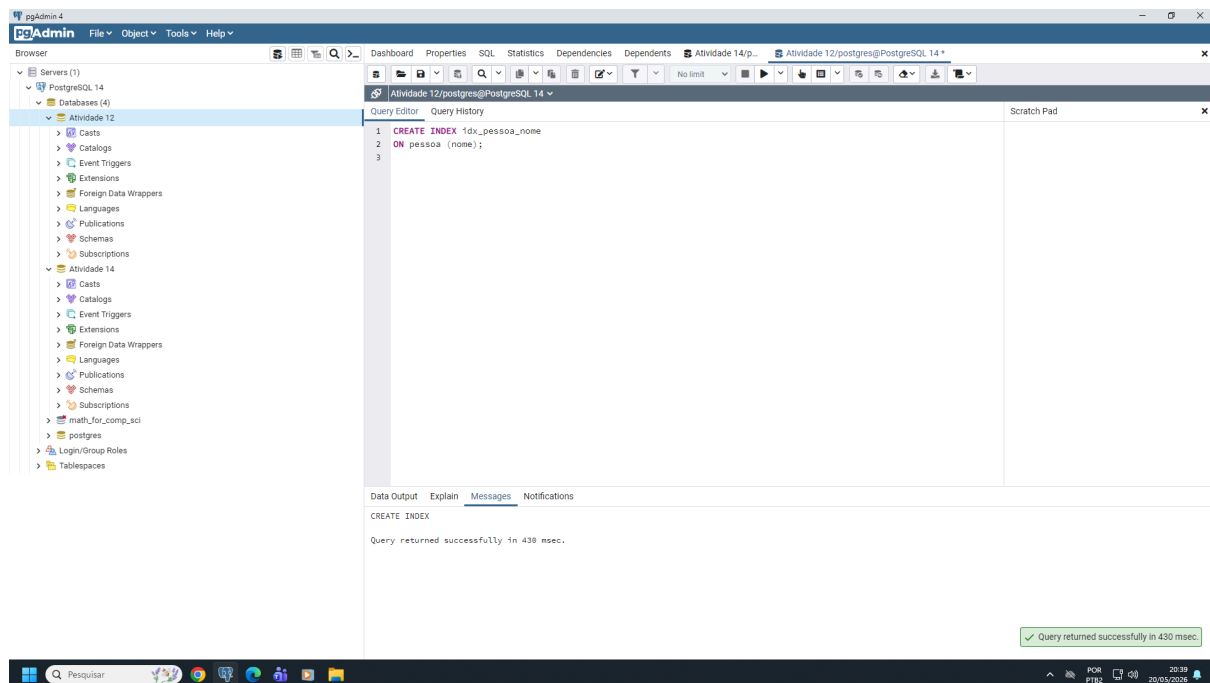
```
WHERE nome = 'João Santos';
```



PARTE 3:

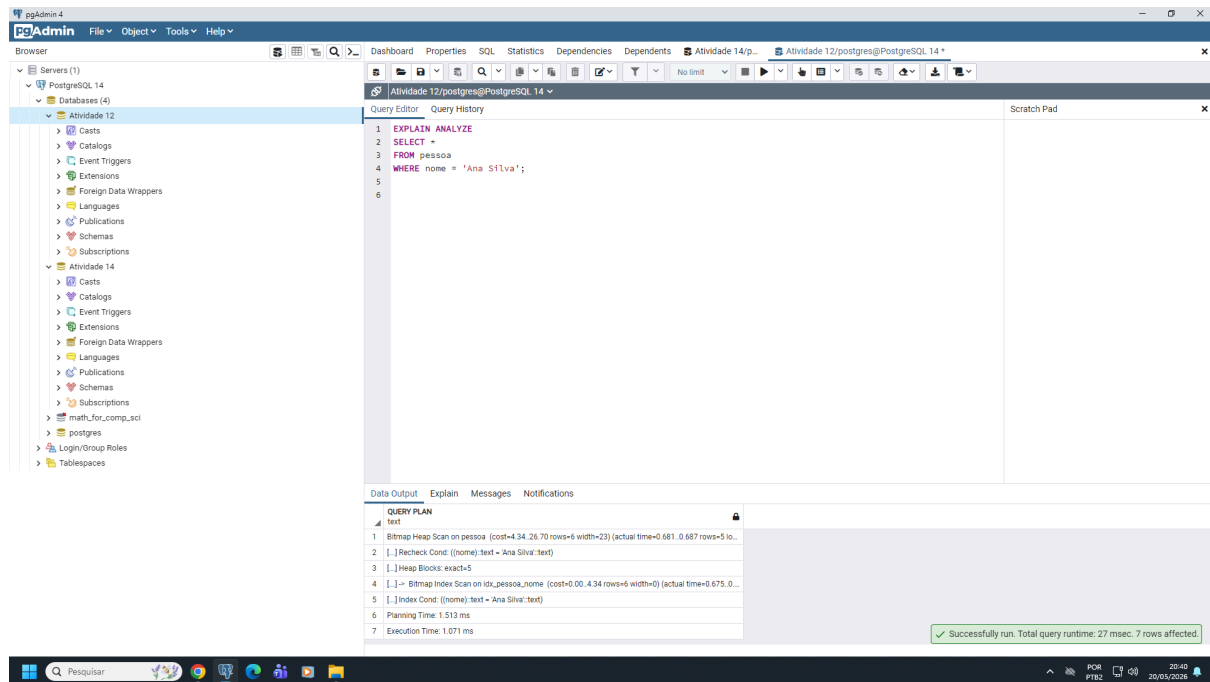
3º Crie um índice B-tree na coluna nome:

```
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```



PARTE C:

4. Execute novamente as duas consultas da Parte A.



pgAdmin 4

File Object Tools Help

Dashboard Properties SQL Statistics Dependencies Dependents Atividade 14/p... Atividade 12/postgres@PostgreSQL 14 *

Browser

Servers (1)

PostgreSQL 14

Databases (4)

Atividade 12

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

Atividade 14

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

math_for_comp_sci

postgres

Login/Group Roles

Tablespaces

Query Editor Query History

Scratch Pad

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM pessoa
4 WHERE nome = 'Ana Silva';
5
6
```

Data Output Explain Messages Notifications

QUERY PLAN

1 Seq Scan on pessoa (cost=4.34..26.70 rows=6 width=23) (actual time=0.681..0.687 rows=5 lo..)

2 [.] Recheck Cond: ((nome)::text = 'Ana Silva':text)

3 [.] Heap Blocks: exact=5

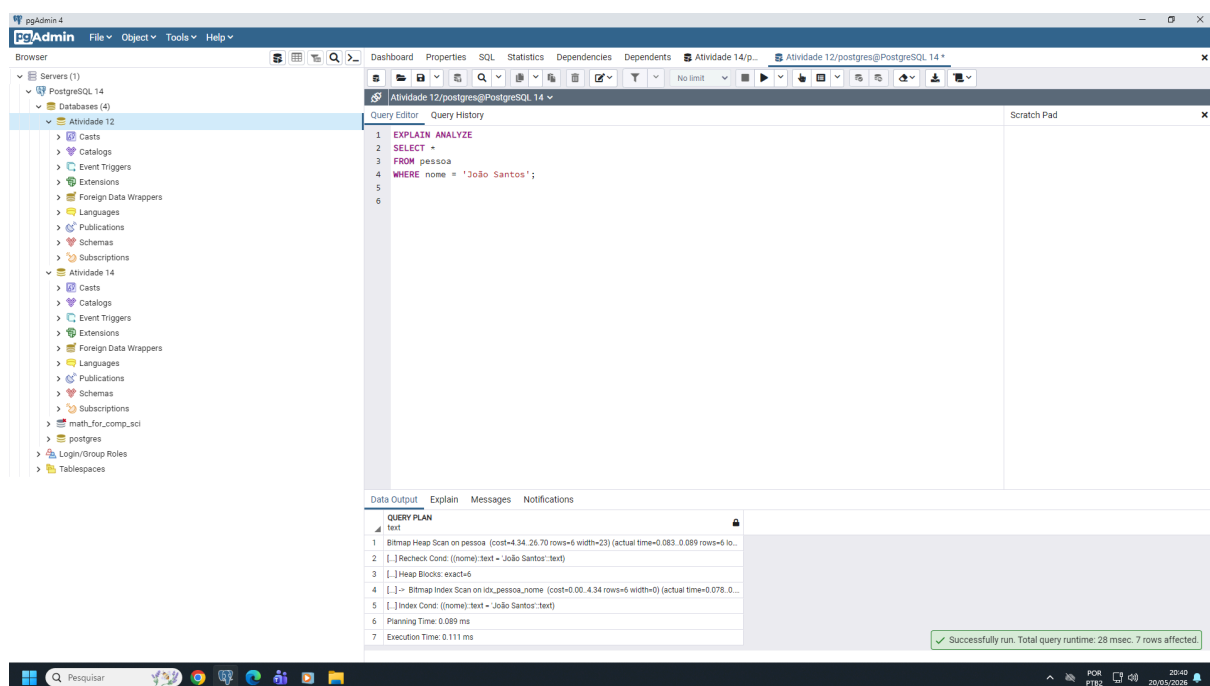
4 [.] Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.675..0..)

5 [.] Index Cond: ((nome)::text = 'Ana Silva':text)

6 Planning Time: 1.513 ms

7 Execution Time: 1.071 ms

Successfully run. Total query runtime: 27 msec. 7 rows affected.



pgAdmin 4

File Object Tools Help

Dashboard Properties SQL Statistics Dependencies Dependents Atividade 14/p... Atividade 12/postgres@PostgreSQL 14 *

Browser

Servers (1)

PostgreSQL 14

Databases (4)

Atividade 12

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

Atividade 14

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

math_for_comp_sci

postgres

Login/Group Roles

Tablespaces

Query Editor Query History

Scratch Pad

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM pessoa
4 WHERE nome = 'João Santos';
5
6
```

Data Output Explain Messages Notifications

QUERY PLAN

1 Bitmap Heap Scan on pessoa (cost=4.34..26.70 rows=6 width=23) (actual time=0.083..0.089 rows=6 lo..)

2 [.] Recheck Cond: ((nome)::text = 'João Santos':text)

3 [.] Heap Blocks: exact=6

4 [.] Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.078..0..)

5 [.] Index Cond: ((nome)::text = 'João Santos':text)

6 Planning Time: 0.089 ms

7 Execution Time: 0.111 ms

Successfully run. Total query runtime: 28 msec. 7 rows affected.

Parte D – Responda, de forma objetiva:

- Qual plano foi utilizado antes do índice?
Seq Scan (Varredura sequencial)
- Qual plano foi utilizado após o índice?
Index Scan ou Bitmap Scan
- Houve redução no tempo de execução?
Sim, houve redução significativa.

• **O índice foi utilizado em ambos os nomes testados? Justifique com base na saída do plano.**

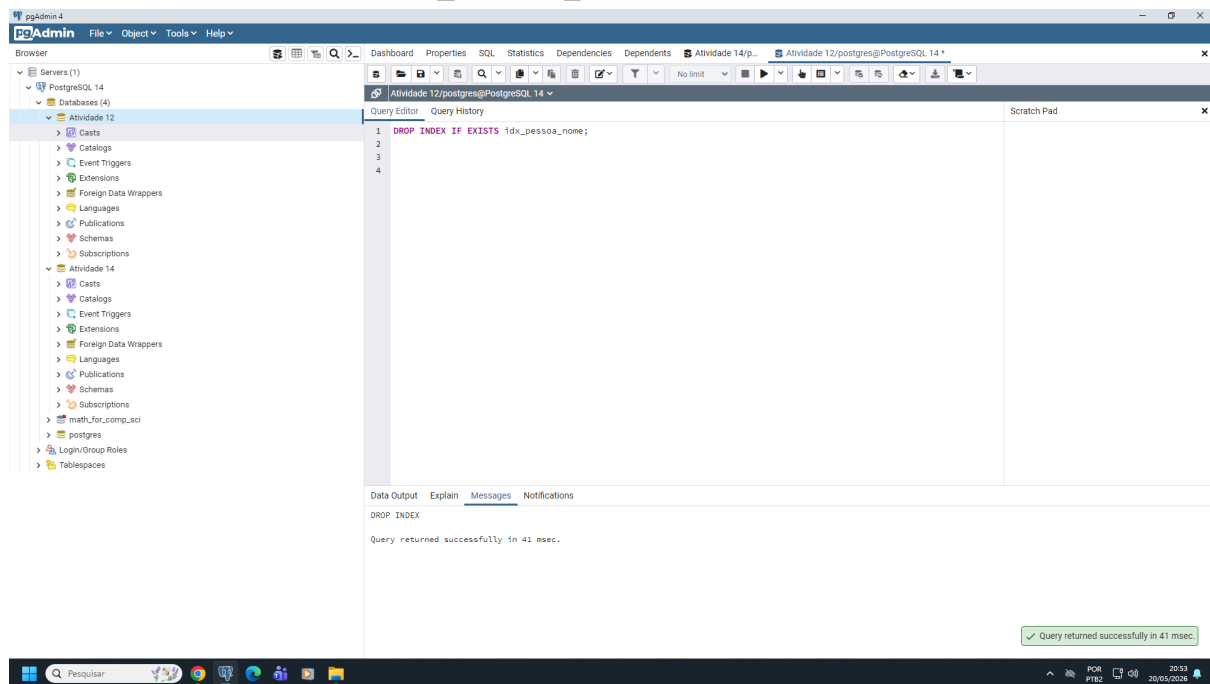
Sim, porque ambos os nomes representam buscas de igualdade de alta seletividade (retornam pouquíssimas linhas dentro do universo de 100 mil).

EXERCÍCIO 2

PARTE A:

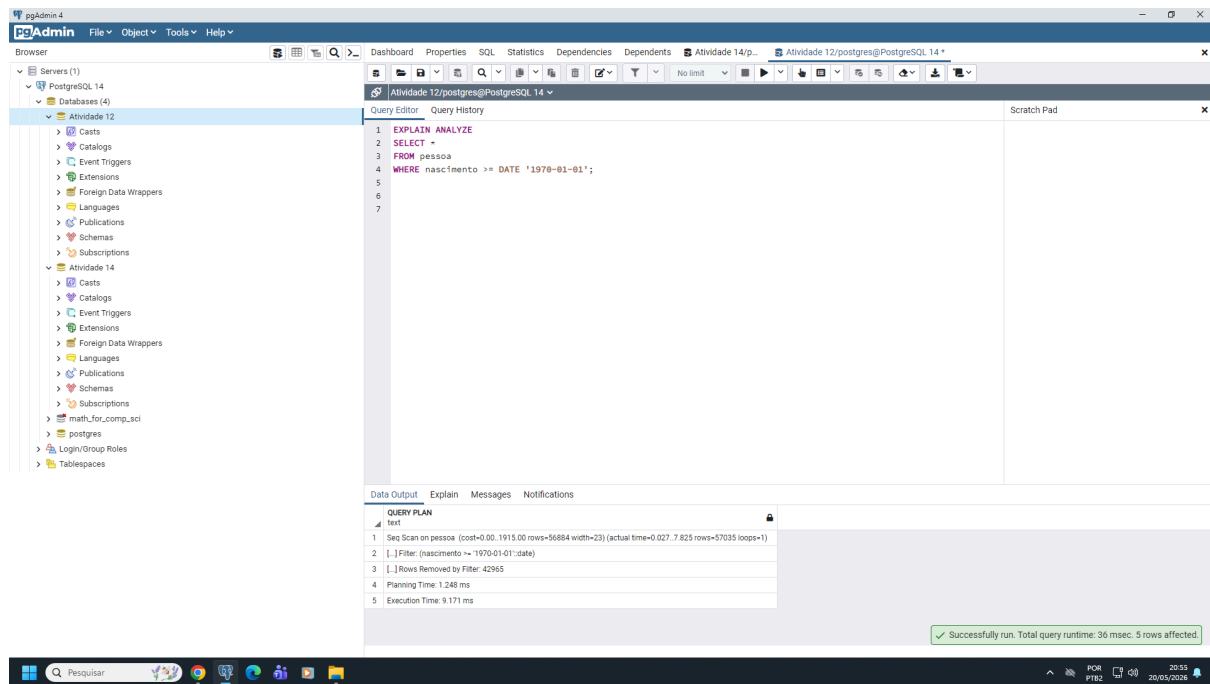
1. Remova o índice criado no exercício anterior (caso ainda exista):

```
DROP INDEX IF EXISTS idx_pessoa_nome;
```



2. Execute a consulta abaixo e observe o plano de execução:

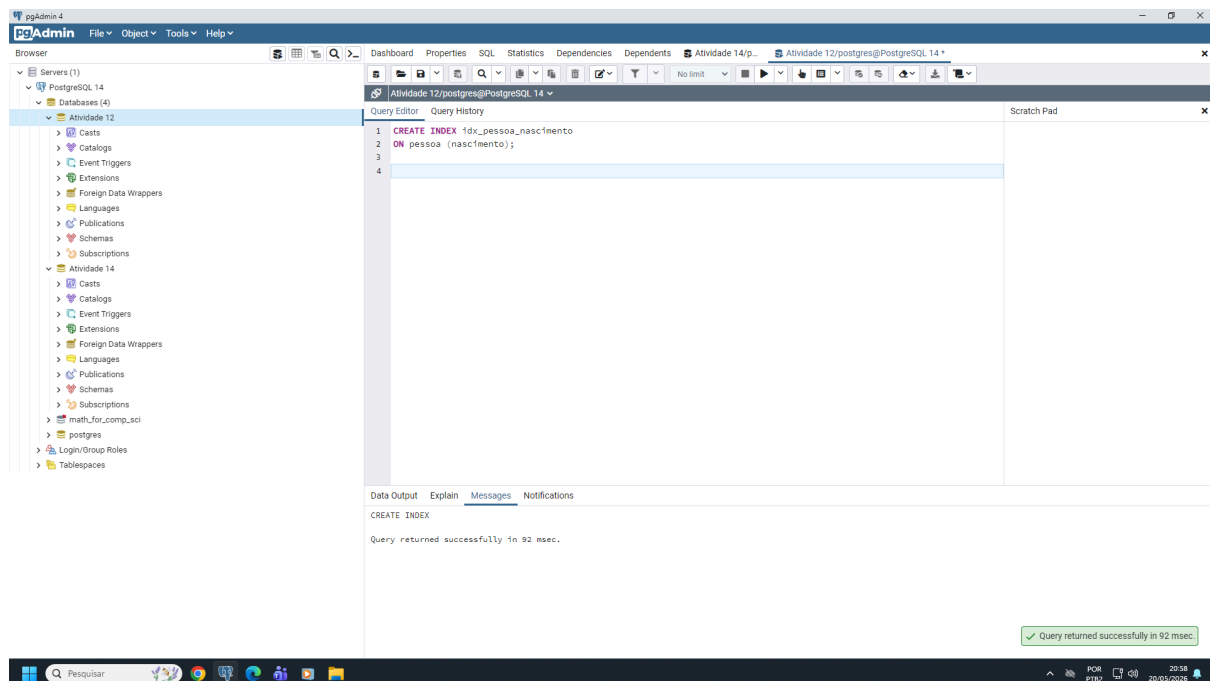
```
EXPLAIN ANALYZE  
SELECT *  
FROM pessoa  
WHERE nascimento >= DATE '1970-01-01';
```



PARTE B:

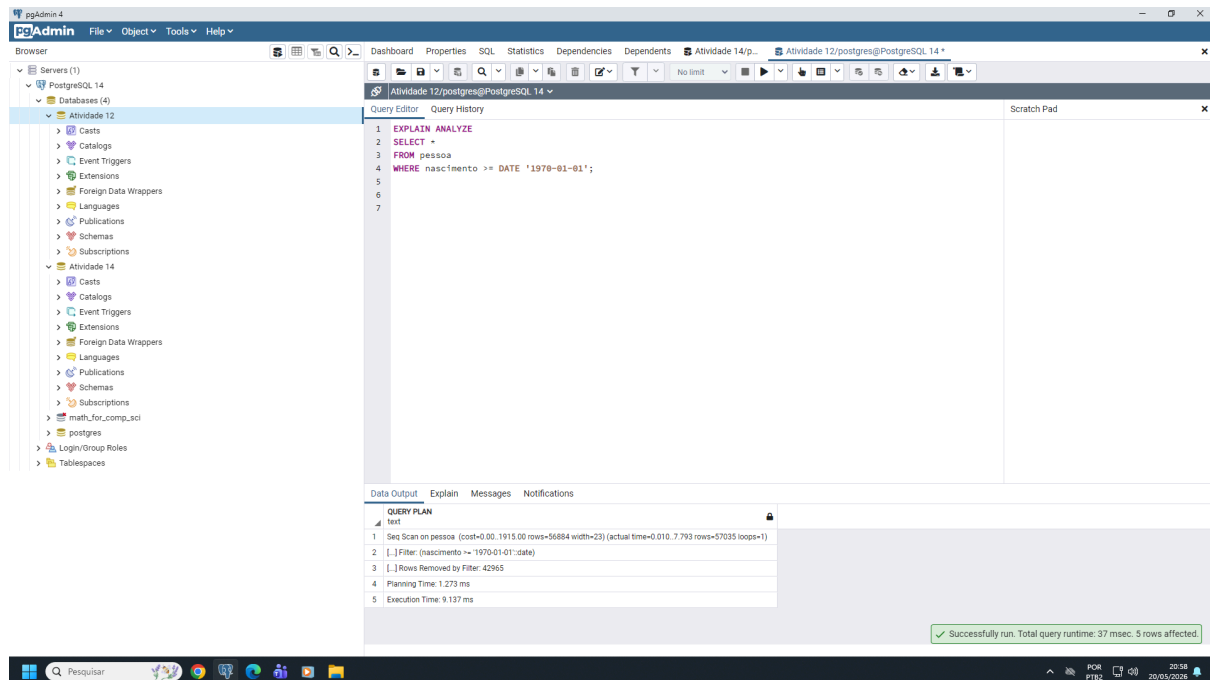
3. Crie um índice B-tree na coluna nascimento:

```
CREATE INDEX idx_pessoa_nascimento
ON pessoa (nascimento);
```



PARTE C:

4. Execute novamente a consulta da Parte A.



Parte D – Responda, de forma objetiva

• O índice criado foi utilizado pelo PostgreSQL?

Não

• O plano de execução mudou após a criação do índice?

Não, continuou como Seq Scan.

• Houve melhora significativa no tempo de execução?

Não.

• Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?

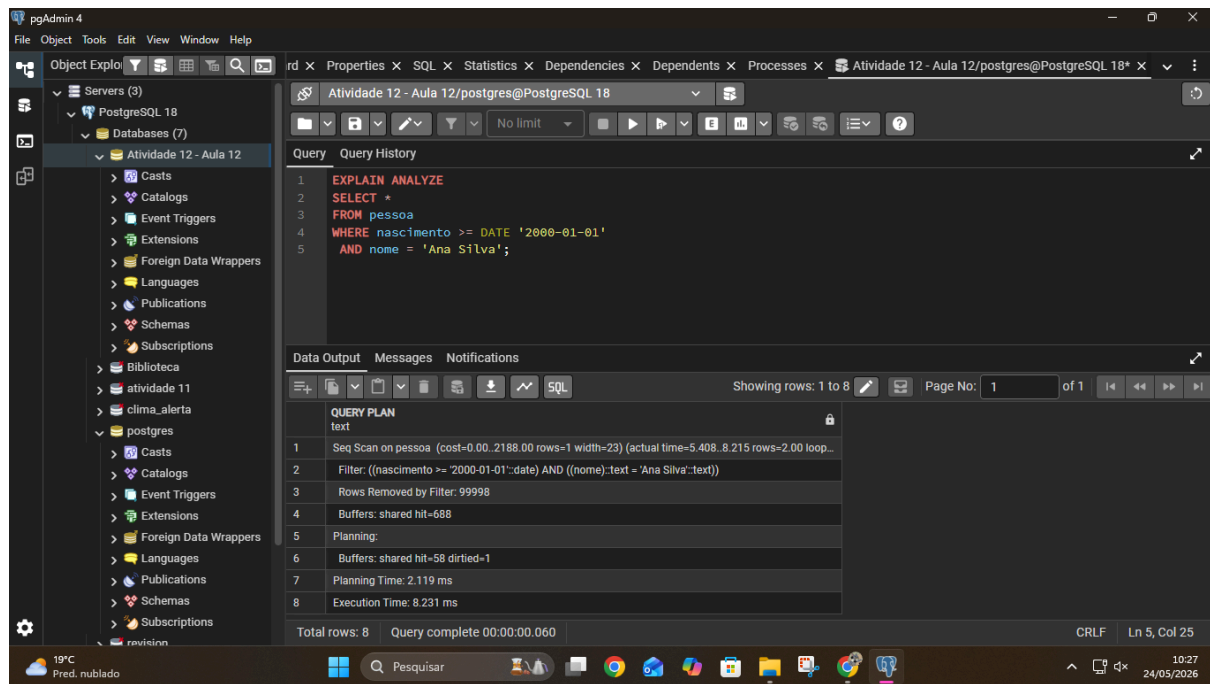
Porque a condição nascimento >= '1970-01-01' possui baixa seletividade (retorna grande parte ou quase a totalidade dos 100 mil registros). Para o PostgreSQL, ler as páginas do índice + ler as páginas da tabela custa mais caro do que ler a tabela direto do início ao fim sequencialmente.

EXERCÍCIO 3

PARTE A:

2. Execute a consulta abaixo e observe o plano de execução:

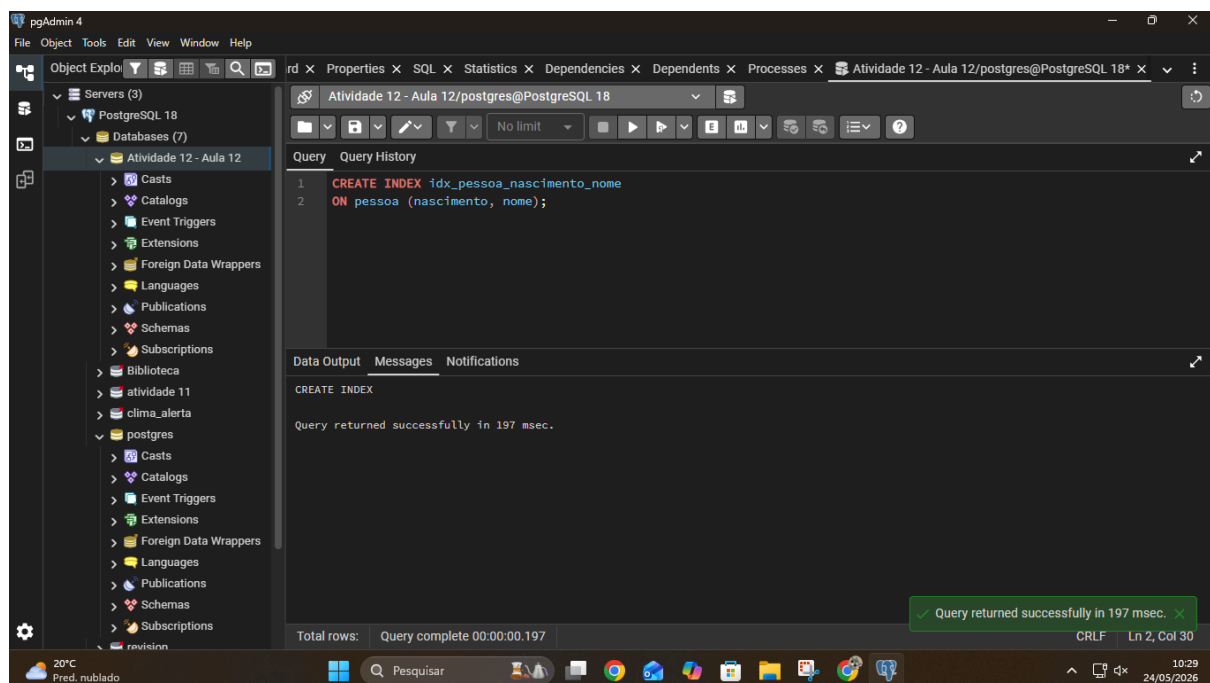
```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
AND nome = 'Ana Silva';
```

PARTEB:

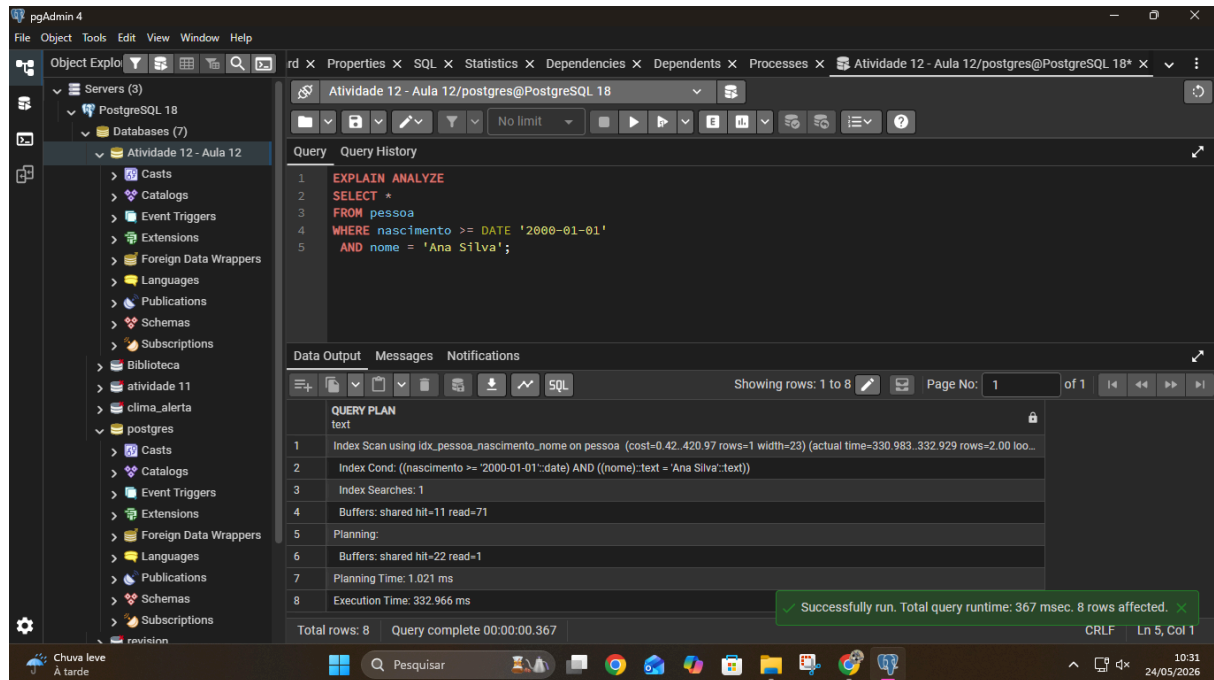
3. Crie um índice composto nas colunas nascimento e nome, nesta ordem:

```
CREATE INDEX idx_pessoa_nascimento_nome
ON pessoa (nascimento, nome);
```



PARTE C:

4. Execute novamente a consulta da Parte A.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including 'Atividade 12 - Aula 12'. The main pane shows the query history and the query plan for the following query:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM pessoa
4 WHERE nascimento >= DATE '2000-01-01'
5 AND nome = 'Ana Silva';
```

The query plan details are as follows:

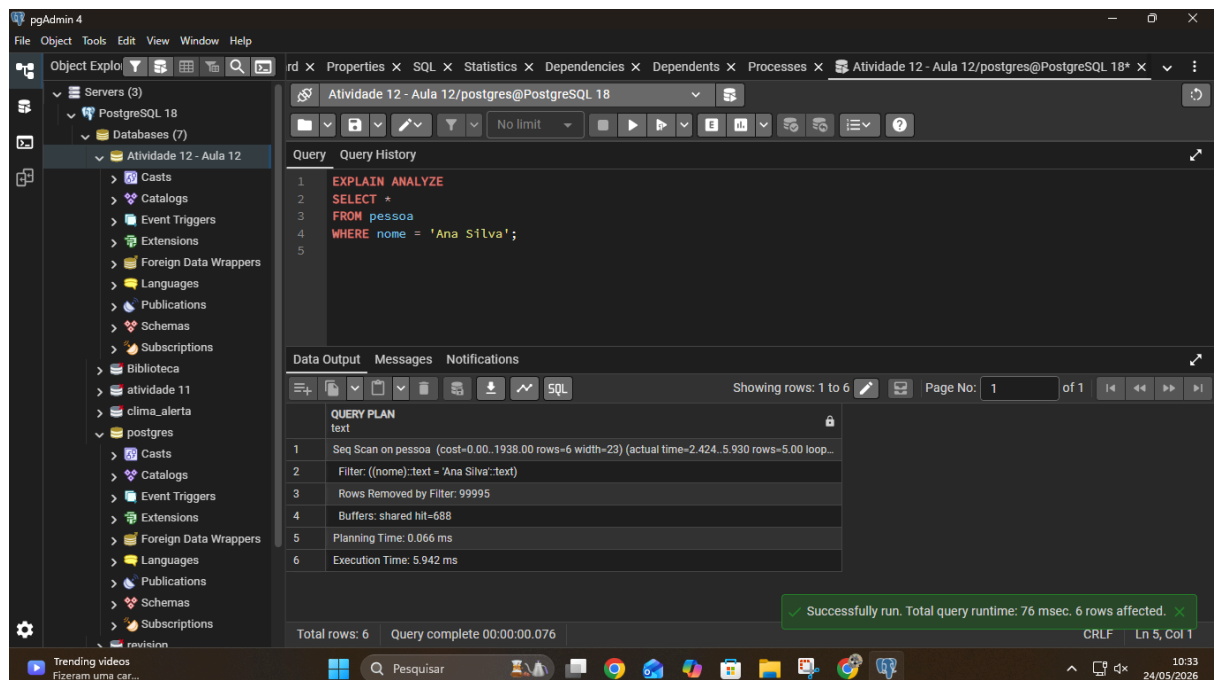
Step	Operation	Cost	Actual Time	Rows	Width
1	Index Scan using idx_pessoa_nascimento_nome on pessoa	0.42	330.983	8	23
2	Index Cond: ((nascimento >= '2000-01-01'::date) AND ((nome)::text = 'Ana Silva'::text))				
3	Index Searches: 1				
4	Buffers: shared hit=11 read=71				
5	Planning:				
6	Buffers: shared hit=22 read=1				
7	Planning Time: 1.021 ms				
8	Execution Time: 332.966 ms				

The status bar indicates: 'Successfully run. Total query runtime: 367 msec. 8 rows affected.' The total rows returned are 8.

PARTE D:

5. Execute a consulta abaixo:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';
```



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including 'Atividade 12 - Aula 12'. The main pane shows the query history and the query plan for the following query:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM pessoa
4 WHERE nome = 'Ana Silva';
5
```

The query plan details are as follows:

Step	Operation	Cost	Actual Time	Rows	Width
1	Seq Scan on pessoa	0.00	2.424	6	23
2	Filter: ((nome)::text = 'Ana Silva'::text)				
3	Rows Removed by Filter: 99995				
4	Buffers: shared hit=688				
5	Planning Time: 0.066 ms				
6	Execution Time: 5.942 ms				

The status bar indicates: 'Successfully run. Total query runtime: 76 msec. 6 rows affected.' The total rows returned are 6.

Parte E – Responda, de forma objetiva

- Qual plano foi utilizado antes da criação do índice composto?

Seq Scan.

- Qual plano foi utilizado após a criação do índice composto?

Index Scan.

- O índice composto foi utilizado na consulta que filtra apenas por nome?

Não.

- Por que a ordem das colunas no índice composto é relevante?

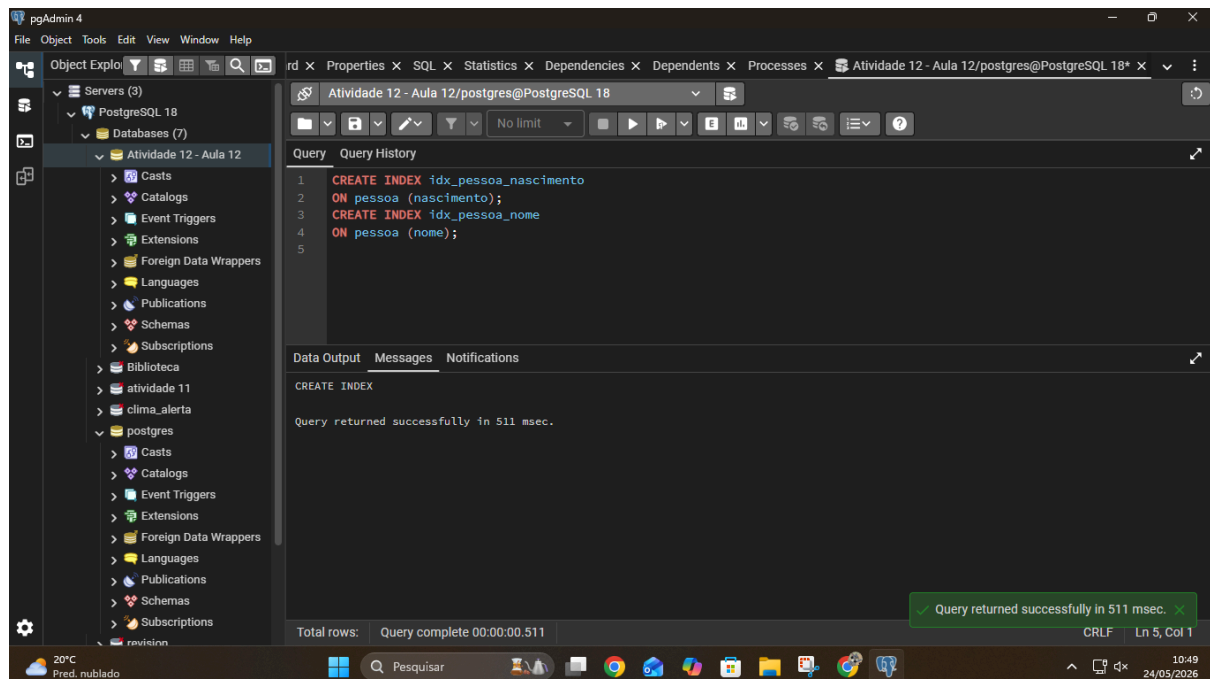
Índices compostos B-tree funcionam estritamente da esquerda para a direita. Como o índice foi criado como (nascimento, nome), o banco não consegue pular a primeira coluna para buscar diretamente pela segunda. O filtro precisa incluir a coluna principal (líder) para ativar a árvore de busca de forma eficiente.

EXERCÍCIO 4

PARTE A:

2. Crie dois índices simples, um para cada coluna:

```
CREATE INDEX idx_pessoa_nascimento  
ON pessoa (nascimento);  
CREATE INDEX idx_pessoa_nome  
ON pessoa (nome);
```



PARTE B:

3. Execute a consulta abaixo e observe atentamente o plano de execução:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01' AND nome = 'Ana Silva';
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including 'atividade 12' and 'postgres'. The main window shows the SQL query and its execution plan. The query is:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM pessoa
4 WHERE nascimento >= DATE '2000-01-01'
5 AND nome = 'Ana Silva';
```

The execution plan (QUERY PLAN) is as follows:

Line	Plan
3	Filter: (nascimento >= '2000-01-01'::date)
4	Rows Removed by Filter: 3
5	Heap Blocks: exact=5
6	Buffers: shared hit=5 read=2
7	-> Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.921..0.921 rows=5.00 loop=1)
8	Index Cond: ((nome)::text = 'Ana Silva'::text)
9	Index Searches: 1
10	Buffers: shared read=2
11	Planning:
12	Buffers: shared hit=32 read=2 dirtied=1
13	Planning Time: 30.740 ms
14	Execution Time: 1.013 ms

Total rows: 14 Query complete 00:00:00.051

Parte C – Responda, de forma objetiva

• O PostgreSQL utilizou os dois índices simples na consulta?

Sim, utilizou os dois índices.

• Qual é o papel do operador BitmapAnd no plano de execução?

Ele cruza os resultados em memória dos dois mapeamentos de bits gerados individualmente por cada índice.

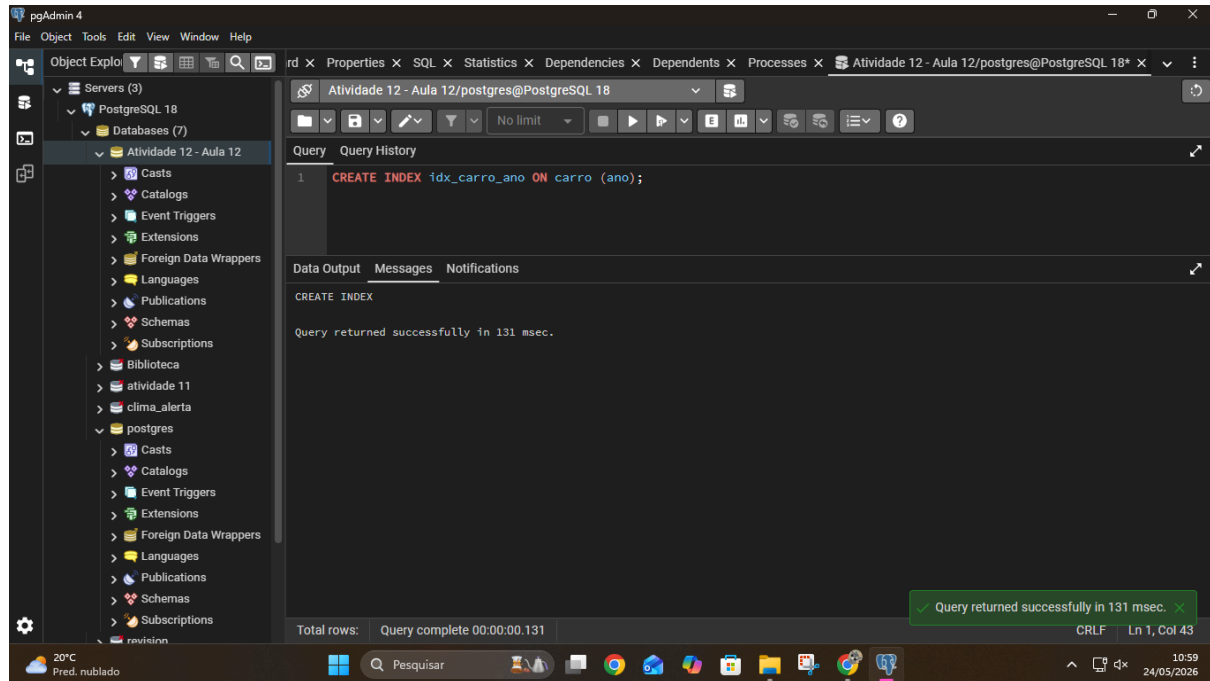
• O que o plano efetivamente fez?

Ele buscou no **idx_pessoa_nascimento** as posições físicas das linhas que atendiam à data, buscou no **idx_pessoa_nome** as posições que atendiam ao nome, aplicou uma operação lógica **AND** (interseção) nessas posições e só então foi à tabela buscar os dados reais dessas linhas

EXERCÍCIO 5

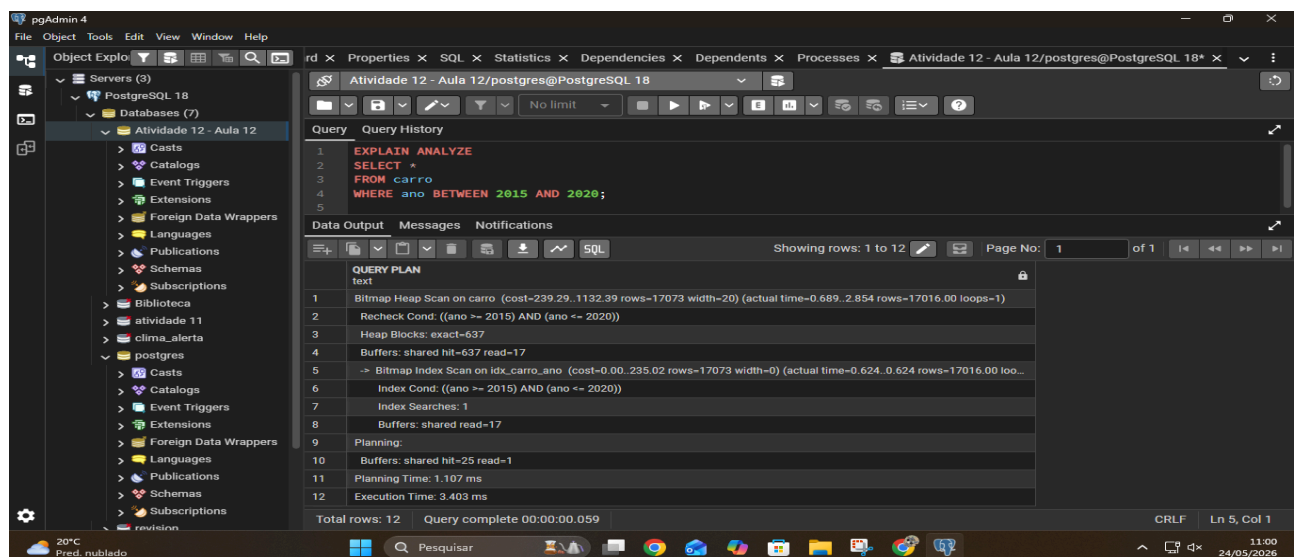
1. Crie o índice mais adequado para esse caso.

```
CREATE INDEX idx_carro_ano ON carro (ano);
```



2. Execute novamente a consulta com EXPLAIN ANALYZE e compare o plano obtido antes e após a criação do índice

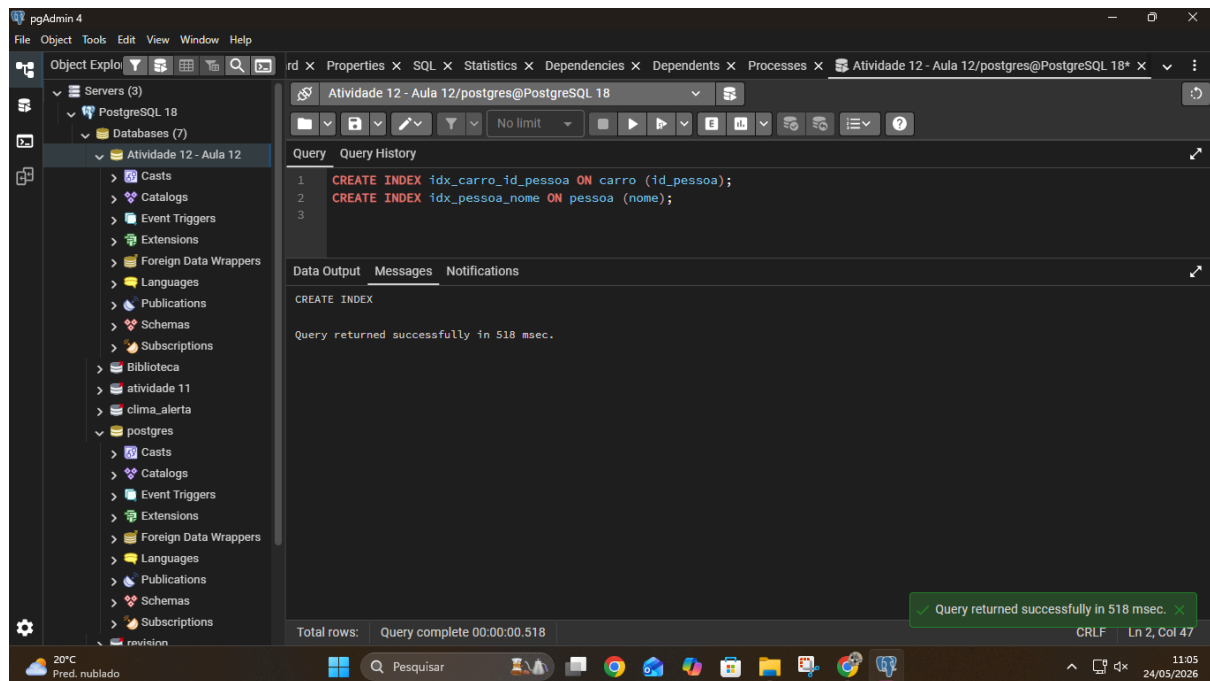
```
EXPLAIN ANALYZE  
SELECT *  
FROM carro  
WHERE ano BETWEEN 2015 AND 2020;
```



EXERCÍCIO 6

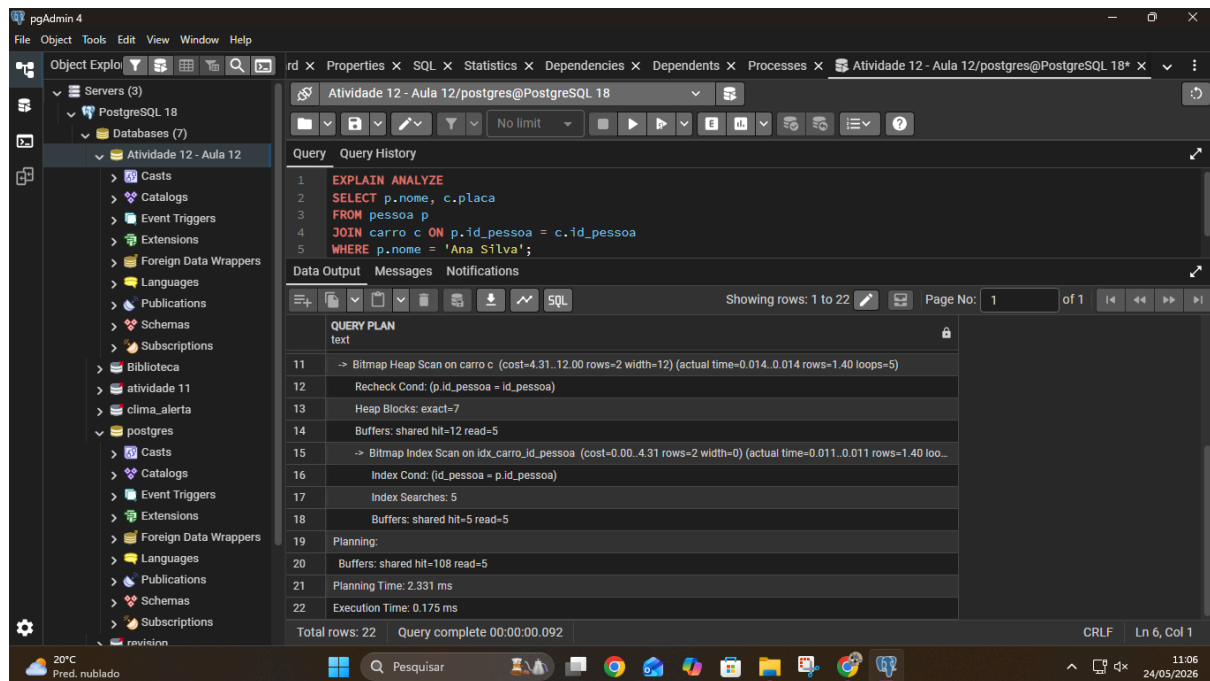
1. Crie os índices necessários para melhorar o desempenho da consulta.

```
CREATE INDEX idx_carro_id_pessoa ON carro (id_pessoa);  
CREATE INDEX idx_pessoa_nome ON pessoa (nome);
```



2. Execute novamente a consulta e análise as alterações no plano de execução.

```
EXPLAIN ANALYZE  
SELECT p.nome, c.placa  
FROM pessoa p  
JOIN carro c ON p.id_pessoa = c.id_pessoa  
WHERE p.nome = 'Ana Silva';
```



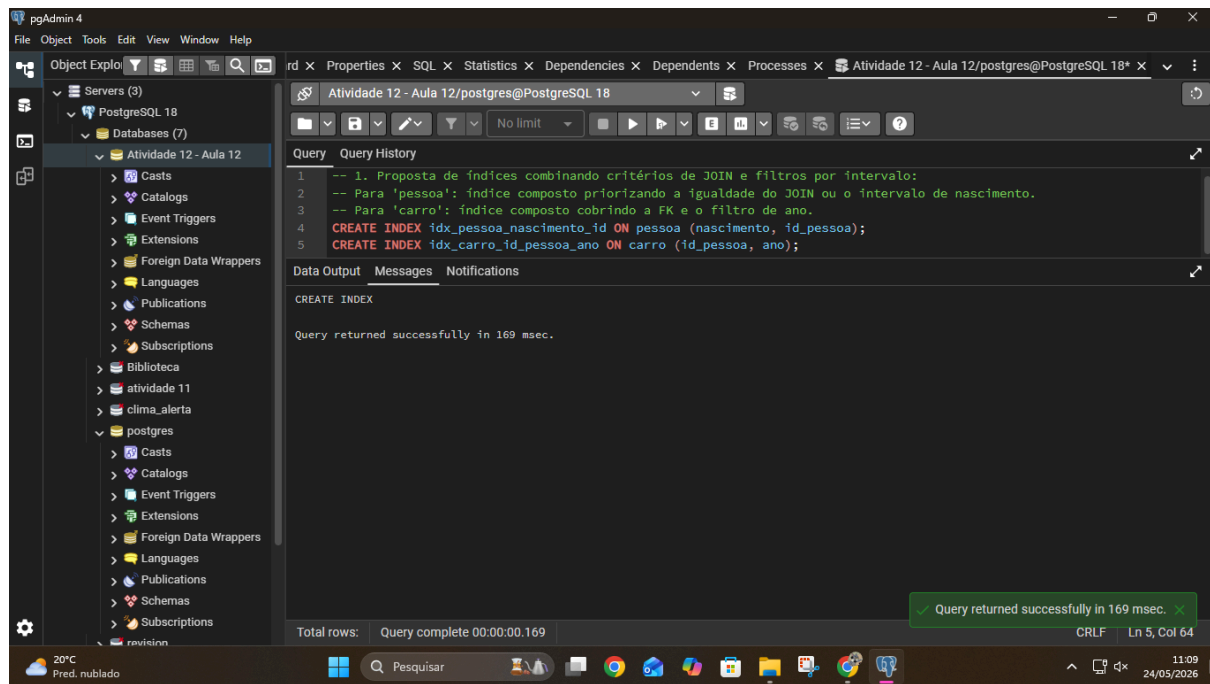
EXERCÍCIO 7

1. Analise quais colunas participam:

- o do JOIN;
- o dos filtros da cláusula WHERE.

Proponha e crie os índices necessários para melhorar o desempenho da consulta. Justifique a escolha entre índices simples ou compostos.

```
-- 1. Proposta de índices combinando critérios de JOIN e filtros
por intervalo:
-- Para 'pessoa': índice composto priorizando a igualdade do JOIN
ou o intervalo de nascimento.
-- Para 'carro': índice composto cobrindo a FK e o filtro de ano.
CREATE INDEX idx_pessoa_nascimento_id ON pessoa (nascimento,
id_pessoa);
CREATE INDEX idx_carro_id_pessoa_ano ON carro (id_pessoa, ano);
```



-- Execução da consulta complexa

```
EXPLAIN ANALYZE
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nascimento >= DATE '1980-01-01'
AND c.ano >= 2018;
```


EXERCÍCIO 8

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a server named 'PostgreSQL 18' with a database 'Atividade 12 - Aula 12'. The main pane shows a SQL query:

```
2 SELECT p.nome, c.placa, c.ano
3 FROM pessoa p
4 JOIN carro c ON p.id_pessoa = c.id_pessoa
5 WHERE p.nascimento >= DATE '1980-01-01'
6 AND c.ano >= 2018;
```

Below the query, the 'Query Plan' is displayed, showing the execution strategy. The plan includes a 'Seq Scan on pessoa p' with a cost of 0.00 and an actual time of 0.008. The plan also shows the 'Filter: (nascimento >= '1980-01-01':date)' and the 'Rows Removed by Filter: 57213'. The total rows returned are 18, and the query completed in 00:00:00.050.

Parte A – Consulta sem índice GiST 1.

Execute a consulta abaixo e observe o plano de execução:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

The screenshot shows the pgAdmin 4 interface with the query executed. The main pane displays the query:

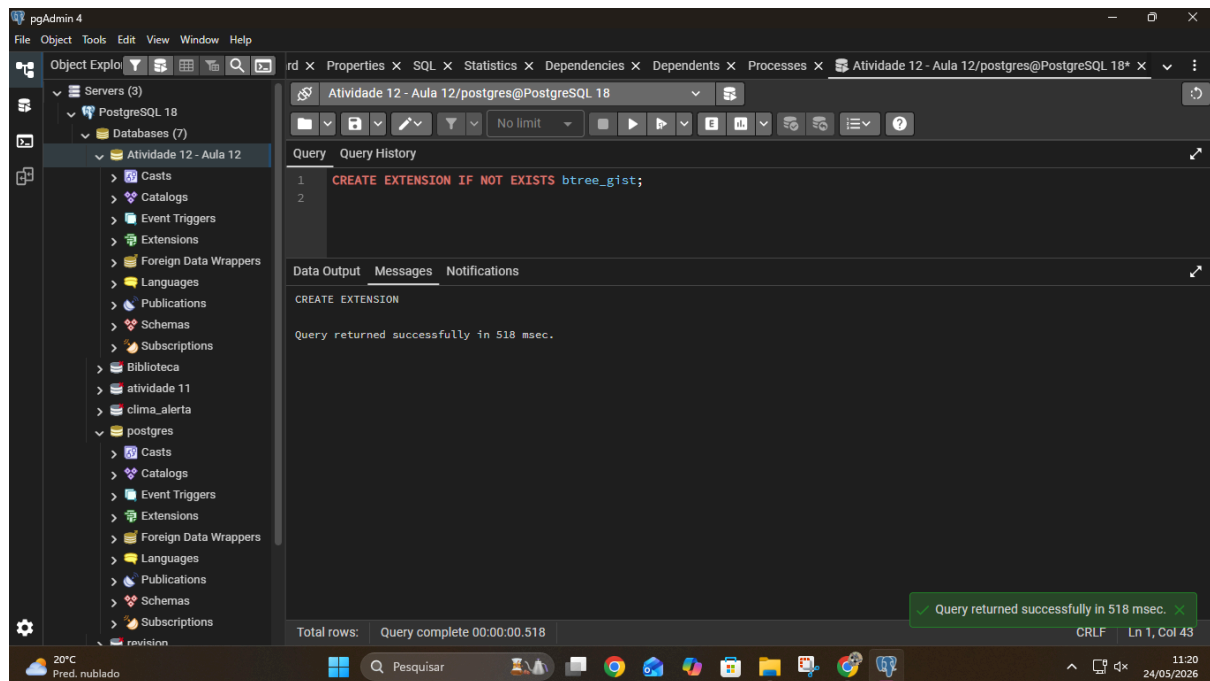
```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM pessoa
4 WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
5
```

Below the query, the 'Query Plan' is displayed, showing the execution strategy. The plan includes a 'Bitmap Heap Scan on pessoa' with a cost of 338.59 and an actual time of 1.323. The plan also shows the 'Recheck Cond: ((nascimento >= '1980-01-01':date) AND (nascimento <= '1990-12-31':date))' and the 'Index Cond: ((nascimento >= '1980-01-01':date) AND (nascimento <= '1990-12-31':date))'. The total rows returned are 12, and the query completed in 00:00:00.069. A green message at the bottom indicates 'Successfully run. Total query runtime: 69 msec. 12 rows affected.'

Parte B – Criação do índice GiST

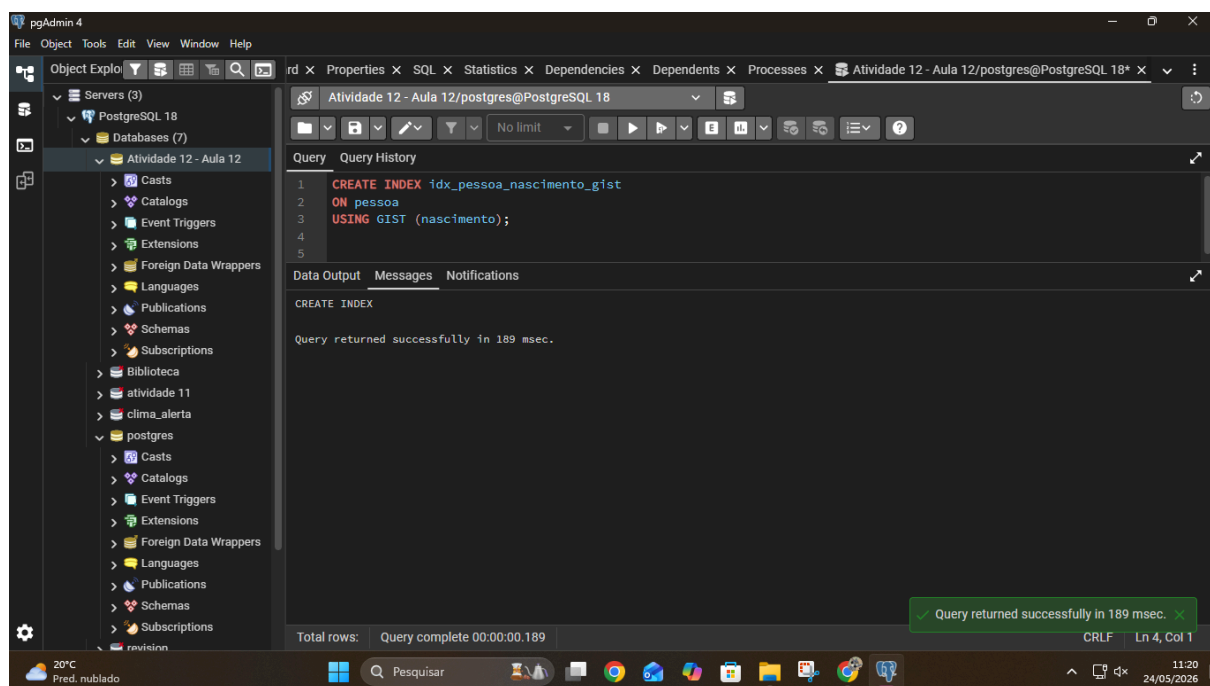
2. Caso ainda não esteja habilitada, ative a extensão necessária para suporte a GiST em tipos escalares:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
```



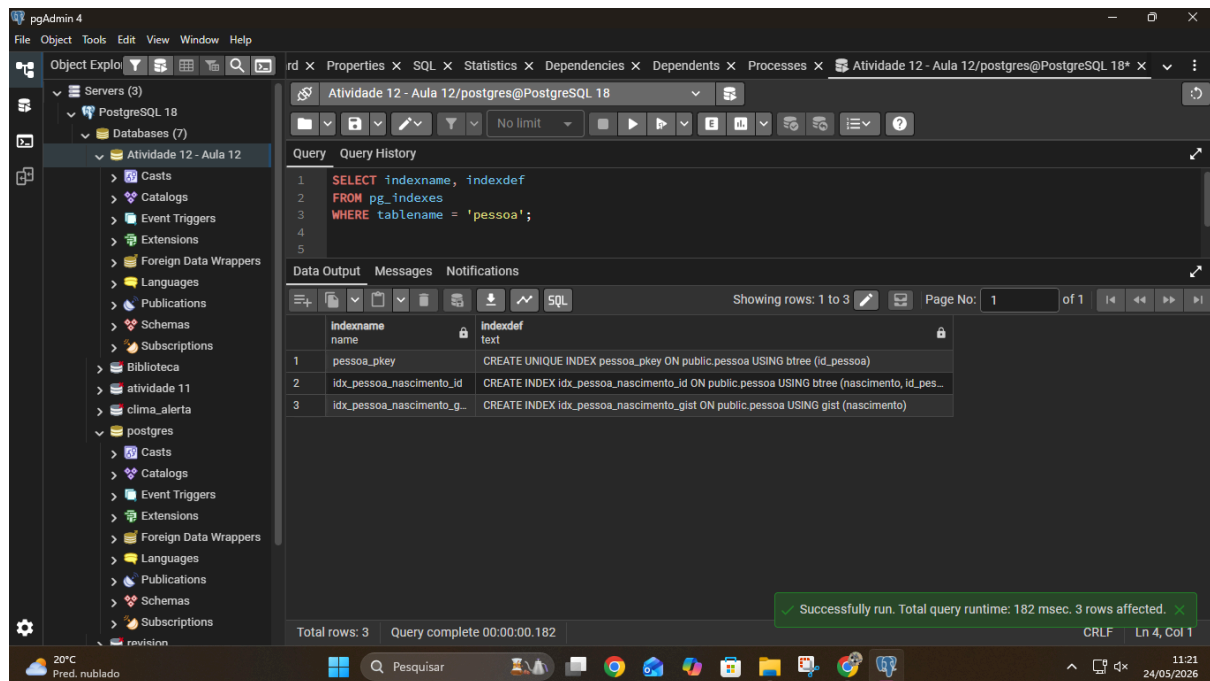
3. Crie um índice do tipo GiST sobre a coluna nascimento:

```
CREATE INDEX idx_pessoa_nascimento_gist  
ON pessoa  
USING GIST (nascimento);
```



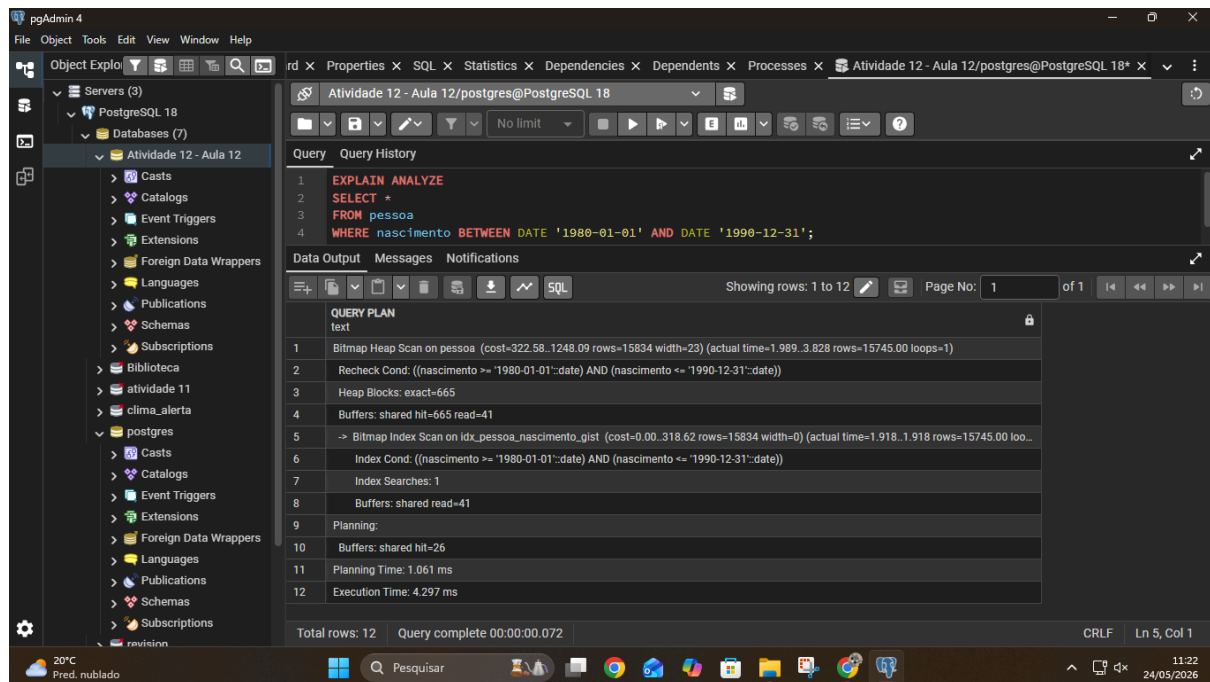
4. Certifique que o um índice foi criado:

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'pessoa';
```



Parte C – Consulta com índice GiST

5. Execute novamente a consulta da Parte A.



Parte D – Responda, de forma objetiva

• Qual plano foi utilizado antes da criação do índice GiST?

Seq Scan.

• O índice GiST foi utilizado após a criação?

Sim (geralmente como Bitmap Index Scan ou Index Scan).

• Houve melhora no tempo de execução?

Sim, pois o filtro delimita um intervalo específico menor (10 anos), tornando a busca no índice vantajosa frente à varredura total.